

2.1. Changer de message avec A et B

Écrire un mot par-dessus un autre le remplace.

1. Ce qu'il faut comprendre

B écrit « APPUIE SUR A », A remet « APPUIE SUR B ». Chaque message **recouvre** le précédent, à la même place.

Les deux font **douze** caractères : le nouveau couvre exactement l'ancien, et il ne reste aucune lettre de trop.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **489 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         68 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles   76 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
LireManette    37 o ← bouton() – lit les huit boutons
Tuiles         128 o ← CopierTuiles – 8 lettres sur 43
"APPUIE SUR B" 12 o ← texte() – un texte gravé
"      "        7 o ← texte() – un texte gravé
"APPUIE SUR B" 12 o ← texte() – un texte gravé
"APPUIE SUR A" 12 o ← texte() – un texte gravé
```

Lettres gravées : ABEIPRSU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 4    44 o sur place, dans main() + EcrireTexte (10 o)
bouton() × 2   10 o sur place, dans main() + LireManette (37 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

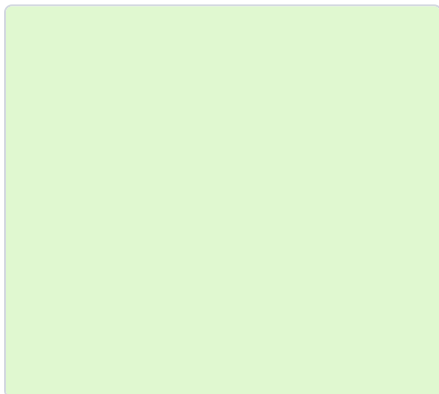
```
#include <texte>    // écrit un texte à l'écran
#include <bouton>    // lit un bouton de la manette
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

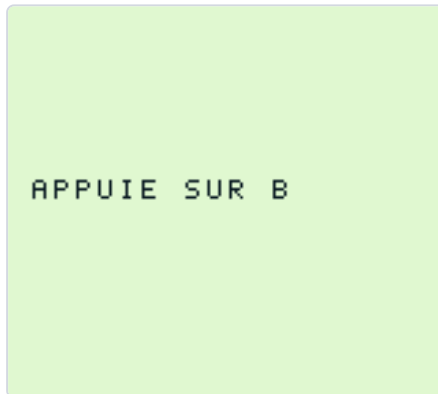
```
int main() {  
  
    texte(1, 8, "APPUIE SUR B");  
    while (true) {  
        image();  
        if (bouton(A)) {  
            texte(6, 4, "        "); // sept espaces, autant que BONJOUR  
            texte(1, 8, "APPUIE SUR B");  
        }  
  
        if (bouton(B)) {  
  
            texte(1, 8, "APPUIE SUR A");  
        }  
    }  
  
    return 0;  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



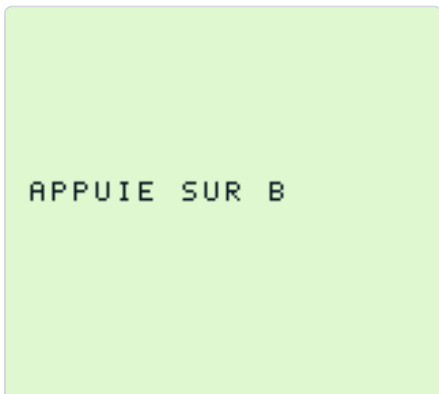
la toute première image — la console vient de démarrer — image 1



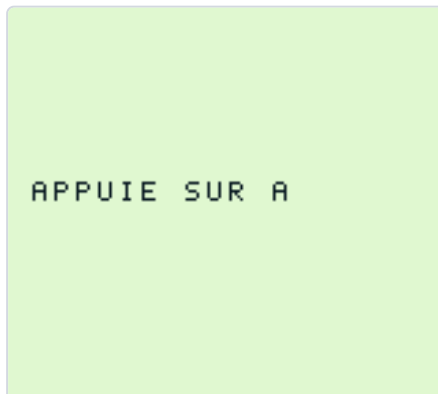
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran n'a pas bougé



B — l'écran a changé

5. Ce qu'on doit voir

B affiche « APPUIE SUR A » ; A remet « APPUIE SUR B ».

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ « APPUIE SUR B » au départ
- ✓ B le remplace par « APPUIE SUR A »
- ✓ A remet « APPUIE SUR B »

Ensuite : **2.2. Effacer ce qu'on a écrit** — chapitre 1 : Les tout premiers pas. · [gameboy3](#) — écrire une cartouche Game Boy en C++